

Lab 2 – Độ quy và chia để trị

1. Mục tiêu

Kết thúc bài thực hành, sinh viên có khả năng:

- Viết được hàm đệ quy với base case và recursive case rõ ràng
- Phân tích độ phức tạp của các hàm đệ quy đơn giản
- Nhận biết “đệ quy chậm” và biết cách tối ưu bằng memoization
- Cài đặt được thuật toán chia để trị cơ bản (merge sort hoặc quick sort)

2. Nội dung

Bài 1 – Viết hàm đệ quy cơ bản

Yêu cầu chung

Tạo file lab2_bai1.py, viết 4 hàm đệ quy sau đây. Mỗi hàm phải có comment ghi rõ:

- Base case là gì
- Recursive case làm gì
- Độ phức tạp Big-O

Hàm 1 – Tính tổng từ 1 đến n

Yêu cầu: Viết hàm `sum_to_n(n)` tính tổng $1 + 2 + 3 + \dots + n$ bằng đệ quy

Hướng dẫn chi tiết:

Bước 1: Xác định base case

- Trường hợp đơn giản nhất: $n = 1 \rightarrow \text{kết quả} = 1$

- Trường hợp $n = 0 \rightarrow$ kết quả = 0

Bước 2: Xác định recursive case

- Tổng từ 1 đến $n = n +$ (tổng từ 1 đến $n-1$)
- Tức là: $\text{sum_to_n}(n) = n + \text{sum_to_n}(n-1)$

Bước 3: Viết code

```
def sum_to_n(n):  
    """  
    Tính tổng 1 + 2 + ... + n bằng đệ quy  
    """  
  
    # Base case: n = 0 hoặc n = 1  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
  
    # Recursive case: n + sum(1..n-1)  
    return n + sum_to_n(n - 1)  
  
# Test  
print(sum_to_n(5)) # Kết quả: 15 (1+2+3+4+5)  
print(sum_to_n(10)) # Kết quả: 55  
  
# Độ phức tạp: O(n)  
# Giải thích: Hàm gọi đệ quy n lần, mỗi lần thực hiện 1 phép cộng
```

Bước 4: Sinh viên tự test với các giá trị khác nhau

- Test với $n = 0$, $n = 1$, $n = 100$
- Kiểm tra kết quả có đúng không (công thức: $n*(n+1)/2$)

Hàm 2 – Tính n mũ k (power)

Yêu cầu: Viết hàm `power(n, k)` tính n^k bằng đệ quy

Hướng dẫn chi tiết:

Bước 1: Xác định base case

- $n^0 = 1$ (bất kỳ số nào mũ 0 đều bằng 1)
- $0^k = 0$ (với $k > 0$)

Bước 2: Xác định recursive case

- $n^k = n \times n^{(k-1)}$
- Ví dụ: $2^5 = 2 \times 2^4 = 2 \times 16 = 32$

Bước 3: Viết code

```
def power(n, k):  
    """  
    Tính  $n^k$  bằng đệ quy  
    """  
    # Base case 1: mũ 0  
    if k == 0:  
        return 1  
  
    # Base case 2: cơ số 0  
    if n == 0:  
        return 0  
  
    # Recursive case:  $n \times n^{(k-1)}$   
    return n * power(n, k - 1)  
  
# Test
```

```
print(power(2, 5)) # Kết quả: 32
```

```
print(power(3, 4)) # Kết quả: 81
```

```
print(power(5, 0)) # Kết quả: 1
```

Độ phức tạp: $O(k)$

Giải thích: Hàm gọi đệ quy k lần

Lưu ý cho sinh viên: Có cách tối ưu hơn bằng "chia để trị" ($O(\log k)$) nhưng không yêu cầu trong bài này.

Hàm 3 – Đảo chuỗi (reverse string)

Yêu cầu: Viết hàm `reverse_string(s)` đảo ngược chuỗi bằng đệ quy

Hướng dẫn chi tiết:

Bước 1: Xác định base case

- Chuỗi rỗng "" → trả về ""
- Chuỗi 1 ký tự → trả về chính nó

Bước 2: Xác định recursive case

- Đảo chuỗi = (đảo phần còn lại) + (ký tự đầu)
- Ví dụ: `reverse("hello") = reverse("ello") + "h" = "olle" + "h" = "olleh"`

Bước 3: Viết code

```
def reverse_string(s):  
    """  
    Đảo ngược chuỗi bằng đệ quy  
    """  
  
    # Base case: chuỗi rỗng hoặc 1 ký tự  
    if len(s) <= 1:
```

```
    return s

# Recursive case: đảo phần còn lại + ký tự đầu
return reverse_string(s[1:]) + s[0]

# Test
print(reverse_string("hello")) # Kết quả: "olleh"
print(reverse_string("python")) # Kết quả: "nohtyp"
print(reverse_string("a"))      # Kết quả: "a"
print(reverse_string(""))       # Kết quả: ""

# Độ phức tạp: O(n)
# Giải thích: n lần gọi đệ quy với n = len(s)
```

Hàm 4 – Kiểm tra palindrome (đọc xuôi ngược như nhau)

Yêu cầu: Viết hàm `is_palindrome(s)` kiểm tra chuỗi có phải palindrome không bằng đệ quy

Hướng dẫn chi tiết:

Bước 1: Xác định base case

- Chuỗi rỗng hoặc 1 ký tự → luôn là palindrome (trả về True)

Bước 2: Xác định recursive case

- So sánh ký tự đầu và cuối:
 - Nếu khác nhau → không phải palindrome (trả về False)
 - Nếu giống nhau → kiểm tra đệ quy phần giữa (bỏ đầu và cuối)

Bước 3: Viết code

```
def is_palindrome(s):  
    """  
    Kiểm tra chuỗi có phải palindrome bằng đệ quy  
    """  
  
    # Base case: chuỗi rỗng hoặc 1 ký tự  
    if len(s) <= 1:  
        return True  
  
    # So sánh ký tự đầu và cuối  
    if s[0] != s[-1]:  
        return False  
  
    # Recursive case: kiểm tra phần giữa  
    return is_palindrome(s[1:-1])  
  
# Test  
print(is_palindrome("racecar")) # True  
print(is_palindrome("madam"))  # True  
print(is_palindrome("hello"))  # False  
print(is_palindrome("a"))      # True  
print(is_palindrome(""))       # True  
  
# Độ phức tạp: O(n)  
# Giải thích: Tối đa n/2 lần so sánh, vẫn là O(n)
```

Bước 4: Sinh viên tự test thêm với:

- "abba"
- "python"
- "noon"

Bài 2 – Tối ưu Fibonacci với memoization

Trong bài này, sinh viên sẽ viết 3 phiên bản Fibonacci và so sánh hiệu suất.

File: lab2_bai2.py

Phần A – Fibonacci đệ quy đơn giản

Bước 1: Copy code sau vào file

```
def fibonacci_naive(n):  
    """  
    Fibonacci đệ quy đơn giản - CHẬM  
     $F(n) = F(n-1) + F(n-2)$   
     $F(0) = 0, F(1) = 1$   
    """  
    if n <= 1:  
        return n  
    return fibonacci_naive(n - 1) + fibonacci_naive(n - 2)  
  
# Test  
print("Fibonacci naive:")  
print(fibonacci_naive(10)) # 55  
print(fibonacci_naive(20)) # 6765 (chậm rồi)  
# print(fibonacci_naive(35)) # Rất chậm! Mất vài giây  
  
# Độ phức tạp:  $O(2^n)$  - rất chậm
```

Phần B – Fibonacci với memoization

Yêu cầu: Viết hàm fibonacci_memo(n, memo={}) sử dụng memoization để tối ưu

Hướng dẫn:

Bước 1: Tạo hàm với tham số memo (dịch để lưu kết quả)

```
def fibonacci_memo(n, memo=None):  
    """  
    Fibonacci với memoization - NHANH  
    """  
  
    # Khởi tạo memo nếu chưa có  
    if memo is None:  
        memo = {}  
  
    # TODO: Kiểm tra n đã có trong memo chưa  
    # Nếu có → trả về ngay memo[n]  
  
    # TODO: Base case  
    # Nếu n <= 1 → lưu vào memo và trả về  
  
    # TODO: Recursive case  
    # Tính fibonacci_memo(n-1) + fibonacci_memo(n-2)  
    # Lưu kết quả vào memo[n]  
    # Trả về memo[n]
```

Bước 2: Sinh viên tự hoàn thành các TODO**Gợi ý cấu trúc:**

```
def fibonacci_memo(n, memo=None):  
    if memo is None:  
        memo = {}  
  
    # Kiểm tra memo  
    if n in memo:  
        return memo[n]
```



```
# Base case
if n <= 1:
    memo[n] = n
    return memo[n]

# Recursive case: sinh viên tự viết
# memo[n] = ...
# return memo[n]
```

Bước 3: Test và so sánh thời gian

```
import time

# Test với n = 35
print("\n--- So sánh hiệu suất ---")

start = time.time()
result1 = fibonacci_naive(30) # Chỉ test n=30 vì 35 quá chậm
time1 = time.time() - start
print(f"Naive F(30) = {result1}, thời gian: {time1:.4f}s")

start = time.time()
result2 = fibonacci_memo(100) # Có thể test với 100!
time2 = time.time() - start
print(f"Memo F(100) = {result2}, thời gian: {time2:.6f}s")
```

Phần C – Fibonacci vòng lặp

Yêu cầu: Viết hàm fibonacci_iterative(n) dùng vòng lặp, không dùng đệ quy

Gợi ý:

- Dùng 2 biến `prev` và `curr` để lưu $F(n-2)$ và $F(n-1)$
- Lặp từ 2 đến n , cập nhật `prev` và `curr`
- Độ phức tạp: $O(n)$ thời gian, $O(1)$ không gian

Sinh viên tự cài đặt hoàn toàn.

Bài 3 – Cài đặt Merge Sort hoặc Quick Sort

Trong bài này, sinh viên chọn 1 trong 2 thuật toán để cài đặt.

File: `lab2_bai3.py`

Lựa chọn A – Merge Sort

Yêu cầu: Viết hàm `merge_sort(arr)` sắp xếp mảng bằng merge sort

Gợi ý khung code:

```
def merge_sort(arr):  
    """  
    Sắp xếp mảng bằng Merge Sort  
    Độ phức tạp:  $O(n \log n)$   
    """  
  
    # Base case: mảng 0 hoặc 1 phần tử  
    if len(arr) <= 1:  
        return arr  
  
    # TODO: Divide - chia đôi mảng  
    mid = len(arr) // 2  
    left = arr[:mid]  
    right = arr[mid:]  
  
    # TODO: Conquer - đệ quy sắp xếp 2 nửa
```

```
left_sorted = merge_sort(left)
right_sorted = merge_sort(right)

# TODO: Combine - trộn 2 nửa đã sắp xếp
return merge(left_sorted, right_sorted)

def merge(left, right):
    """
    Trộn 2 mảng đã sắp xếp thành 1 mảng sắp xếp
    """
    result = []
    i = j = 0

    # TODO: So sánh và chọn phần tử nhỏ hơn
    # while i < len(left) and j < len(right):
    #     if left[i] <= right[j]:
    #         ...
    #     else:
    #         ...

    # TODO: Thêm các phần tử còn lại
    # result.extend(...)

    return result

# Test
arr = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = merge_sort(arr)
```

```
print(f"Mảng sau khi sắp xếp: {sorted_arr}")  
# Kết quả: [11, 12, 22, 25, 34, 64, 90]
```

Hướng dẫn cho sinh viên:

1. Hàm `merge_sort`:

- Chia mảng thành 2 nửa
- Gọi đệ quy sắp xếp 2 nửa
- Trộn 2 nửa đã sắp xếp

2. Hàm `merge`:

- Dùng 2 con trỏ `i`, `j` cho `left` và `right`
- So sánh `left[i]` và `right[j]`, chọn nhỏ hơn
- Sau khi hết 1 mảng, thêm phần còn lại của mảng kia

Lựa chọn B – Quick Sort

Yêu cầu: Viết hàm `quick_sort(arr)` sắp xếp mảng bằng quick sort

Gợi ý khung code:

```
def quick_sort(arr):  
    """  
    Sắp xếp mảng bằng Quick Sort  
    Độ phức tạp trung bình:  $O(n \log n)$   
    """  
  
    # Base case  
    if len(arr) <= 1:  
        return arr  
  
    # TODO: Chọn pivot (có thể chọn phần tử giữa)
```

```
pivot = arr[len(arr) // 2]

# TODO: Partition - chia thành 3 phần
# left: các phần tử < pivot
# middle: các phần tử = pivot
# right: các phần tử > pivot

left = [x for x in arr if x < pivot]
middle = [x for x in arr if x == pivot]
right = [x for x in arr if x > pivot]

# TODO: Đệ quy và kết hợp
return quick_sort(left) + middle + quick_sort(right)

# Test
arr = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = quick_sort(arr)
print(f"Mảng sau khi sắp xếp: {sorted_arr}")
```

Hướng dẫn cho sinh viên:

1. Chọn pivot (phần tử chốt)
2. Chia mảng thành 3 phần:
 - Nhỏ hơn pivot
 - Bằng pivot
 - Lớn hơn pivot
3. Đệ quy sắp xếp phần trái và phải
4. Ghép lại: left + middle + right

Phần nâng cao

1. Đo thời gian thực thi với mảng lớn:

```
import time
import random

# Tạo mảng ngẫu nhiên
arr_small = [random.randint(1, 1000) for _ in range(100)]
arr_large = [random.randint(1, 10000) for _ in range(5000)]

# Đo thời gian với mảng nhỏ
start = time.time()
sorted_small = merge_sort(arr_small.copy()) # hoặc quick_sort
time_small = time.time() - start
print(f"Thời gian sắp xếp 100 phần tử: {time_small:.6f}s")

# Đo thời gian với mảng lớn
start = time.time()
sorted_large = merge_sort(arr_large.copy())
time_large = time.time() - start
print(f"Thời gian sắp xếp 5000 phần tử: {time_large:.6f}s")
```

2. So sánh với Python built-in sort:

```
start = time.time()
sorted_builtin = sorted(arr_large)
time_builtin = time.time() - start
print(f"Thời gian Python sorted(): {time_builtin:.6f}s")
```